

ZYVORAI LABS · CUSTOMER DOCUMENTATION



Getting Started

2026-08-29

Prerequisites

- Linux host with Netlink
- One of: `systemd-networkd`, NetworkManager, or `dhcclient`
- Root/ `sudo` to install the binary and unit
- Optional: Rust toolchain to build from source

1. Install

Release tarball (recommended):

```
curl -LO https://github.com/zyvorai/netevd/releases/latest/download/netevd-linux-amd64.tar.gz
tar xzf netevd-*-linux-amd64.tar.gz && cd netevd-*-linux-amd64
sudo ./install.sh
sudo systemctl enable --now netevd
```

From source:

```
git clone https://github.com/zyvorai/netevd.git && cd netevd
cargo build --release
sudo install -Dm755 target/release/netevd /usr/bin/netevd
sudo install -Dm644 systemd/netevd.service /lib/systemd/system/netevd.service
sudo install -Dm644 config/netevd.example.yaml /etc/netevd/netevd.yaml
sudo systemctl enable --now netevd
```

2. First hook

```
sudo tee /etc/netevd/routable.d/01-notify.sh >/dev/null <<'EOF'
#!/bin/bash
logger -t netevd "Interface $LINK is routable. $ADDRESSES"
0:
sudo chmod +x /etc/netevd/routable.d/01-notify.sh
```

Prefer `routable.d` when you need L3 connectivity; use `carrier.d` only for link-layer up.

3. Verify

```
systemctl status netevd
journalctl -u netevd -f
netevd validate
netevd status
curl -sf http://127.0.0.1:9090/api/v1/status | jq .
curl -sf http://127.0.0.1:9090/health | jq .
```

Operate from CLI

1. Confirm unit active: `systemctl is-active netevd`
2. Validate YAML before edits: `netevd validate -c /etc/netevd/netevd.yaml`
3. Smoke-test API: `netevd status -f json`
4. Bounce an interface and tail events: `netevd events -f -i eth0 -t routable`
5. Remote host: `ssh root@<host> netevd status --endpoint http://127.0.0.1:9090`

Troubleshooting

Symptom	Check
Hook never fires	Executable bit; correct <code>*.d</code> directory; filters in YAML
No API on <code>:9090</code>	<code>api.enabled</code> / bind address in <code>netevd.yaml</code> ; unit active
No events	Backend matches host (<code>systemd-networkd</code> vs NM vs dhclient)

Next

- [Using the operator surfaces](#)
 - [Admin basics](#)
 - [Workflows](#)
-

netevd has **no shipped browser console** for day-2 work. Operators use hook directories, the CLI, REST/Prometheus, and systemd journals. A static dashboard HTML exists in the repo for future embedding but is not the primary surface.

Mental model

Where to look

Surface	Role
/etc/netevd/*.d/	Event scripts (primary UX)
/etc/netevd/netevd.yaml	Backend, filters, API bind, metrics
netevd status list show events	Live inspection via API
journalctl -u netevd	Daemon + hook exit logs
:9090 / :9091	Automation scrape / status on <host>

Operate from CLI

1. **Hooks first:** add executable scripts under the event directory that matches your backend ([Page guides](#)).
2. **Validate:** `netevd validate -c /etc/netevd/netevd.yaml`
3. **Observe:** `netevd events -f` while bouncing a link (`ip link set eth0 down/up`).
4. **Inspect state:** `netevd list interfaces` , `netevd list rules` , `netevd status -f json` .
5. **Fleet:** scrape `http://<host>:9091/metrics` or curl `http://<host>:9090/api/v1/status` over SSH tunnel.

Tips

1. Start with one `routable.d` logger before complex automation.
2. Use `netevd events -f` while bouncing a link to confirm events.
3. On multi-homed hosts, confirm policy rules with `netevd list rules` or `ip rule`.
4. Keep hooks small and executable; non-zero exits are logged and do not stop other scripts.

Next: [Page-by-page guides](#) · [Workflows](#)

A. First routable notify

1. Install and start `netevd` .
2. Drop an executable script in `/etc/netevd/routable.d/` .
3. Bounce the interface or renew DHCP.
4. Confirm `journalctl -u netevd` , `netevd events --tail 10` , and script side effects.

```
sudo systemctl enable --now netevd
sudo chmod +x /etc/netevd/routable.d/01-notify.sh
sudo ip link set eth0 down && sudo ip link set eth0 up
journalctl -u netevd -n 20 --no-pager
```

B. Multi-homed policy routing

1. List secondary interfaces under `routing.policy_rules` in YAML.
2. Acquire addresses on each uplink.
3. Verify:

```
netevd validate
sudo systemctl restart netevd
netevd list rules
ip rule list
ip route show table $(/200 + $(cat /sys/class/net/eth1/ifindex))
```

Traffic sourced from an address on `eth1` should leave via `eth1`'s gateway.

C. Watch events live

```
netevd events
```

```
or filter:
```

```
etevd events -f -i eth0 -t routable
```

D. Fleet observability

1. Set `api.bind_address: "0.0.0.0"` only on trusted management networks (or scrape via SSH tunnel).
2. Point Prometheus at `http://<host>:9091/metrics` and/or REST `http://<host>:9090/api/v1/status` .
3. Alert when `netevd_script_executions_total` goes quiet (daemon likely down).

E. Safe change window

```
netevd validate -c /etc/netevd/netevd.yaml  
sudo systemctl restart netevd  
# or when not reload is available  
netevd reload --endpoint http://127.0.0.1:9090/  
journalctl -u netevd -n 50 --no-pager  
netevd status
```

F. Ignore noisy interfaces

Add a YAML filter with `action: ignore` for `docker*`, `veth*`, etc. (see `config/netevd.example.yaml`), then:

```
netevd validate && sudo systemctl restart netevd  
netevd events -f # confirm docker events no longer dispatch hooks
```

Operate from CLI

Every workflow above is CLI-first: hooks in `/etc/netevd/` , validation via `netevd validate` , observation via `netevd events` and `journalctl` , fleet checks via curl to `<host>:9090` / `:9091` . See [Page-by-page guides](#) for command-level detail per surface.